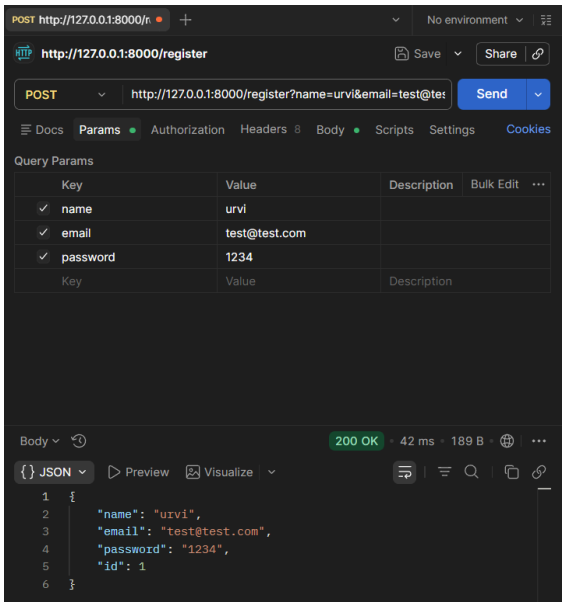
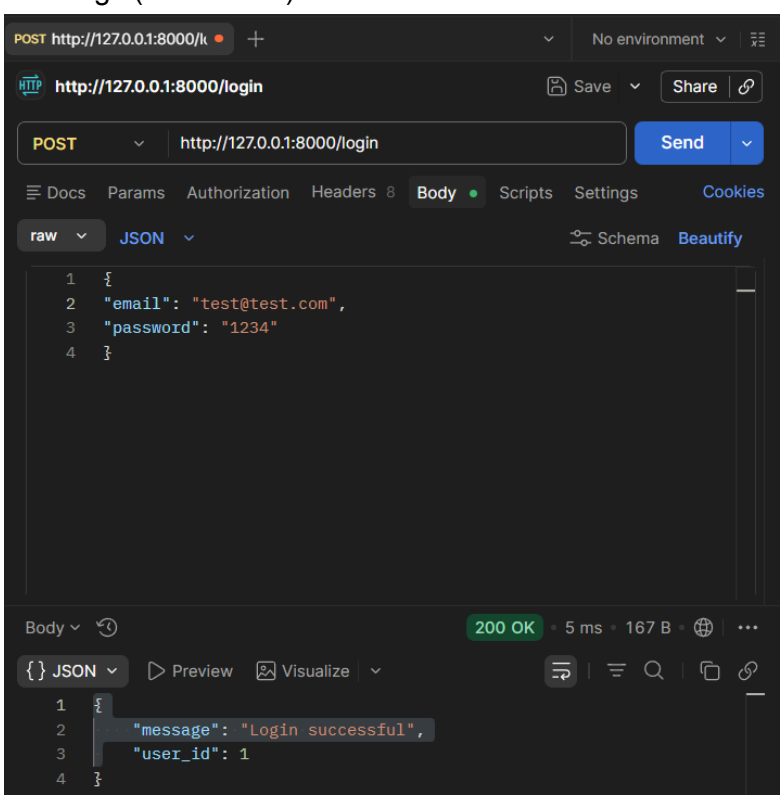
# POSTMAN APIs ENDPOINTS TESTING

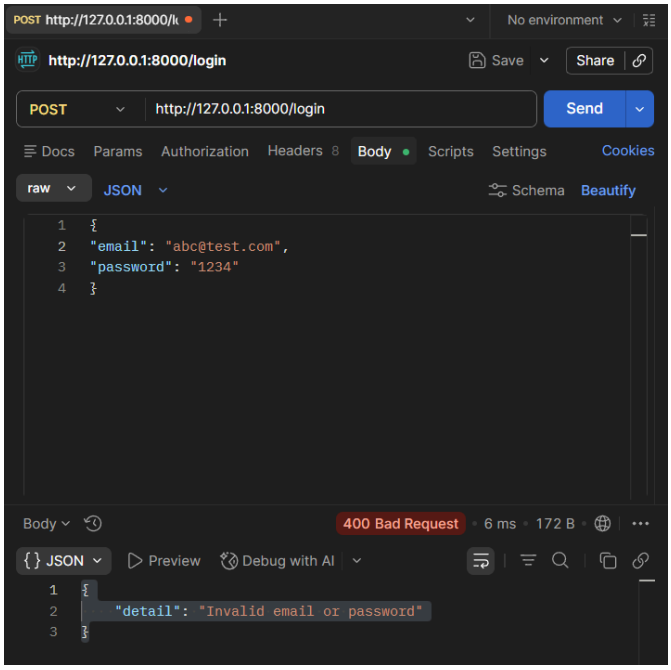
Test register User:



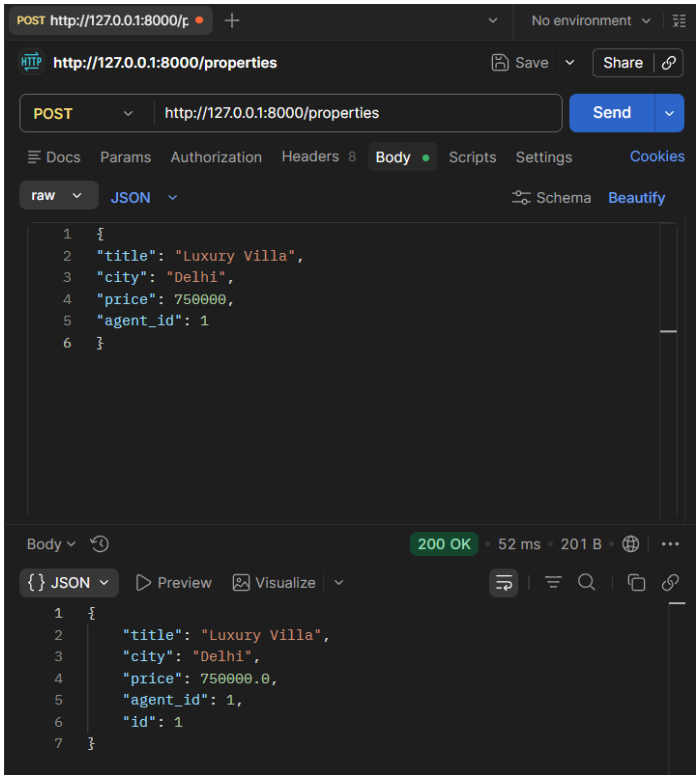

Test Login(Successful) :



Test login(Failed):




Create properties:



Get all properties:

GET http://127.0.0.1:8000/pr

No environment

http://127.0.0.1:8000/properties

Save Share

GET http://127.0.0.1:8000/properties Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Body 200 OK 26 ms 515 B

{ } JSON Preview Visualize


```
[
    {
        "title": "Luxury Villa",
        "city": "Delhi",
        "price": 750000.0,
        "agent_id": 1,
        "id": 1
    },
    {
        "title": "Luxury Villa",
        "city": "Delhi",
        "price": 750000.0,
        "agent_id": 1,
        "id": 2
    },
    {
        "title": "Modern Apartment",
        "city": "Mumbai",
        "price": 450000.0,
        "agent_id": 1,
        "id": 3
    },
    {
        "title": "Beach House",
        "city": "Goa",
        "price": 950000.0,
        "agent_id": 1,
        "id": 4
    },
    {
```


Search property:

GET http://127.0.0.1:8000/pr

No environment

http://127.0.0.1:8000/properties/search/

Save Share

GET http://127.0.0.1:8000/properties/search/?city=Delhi Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

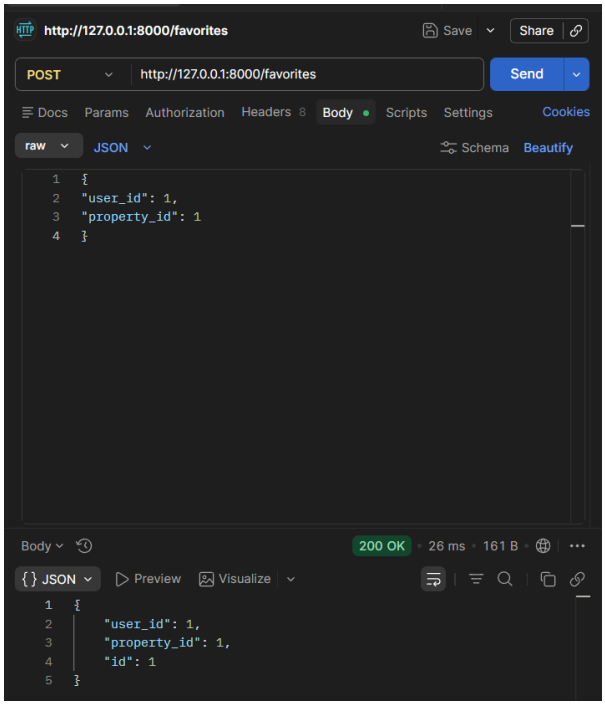
raw JSON Schema Beautify

Body 200 OK 4 ms 281 B

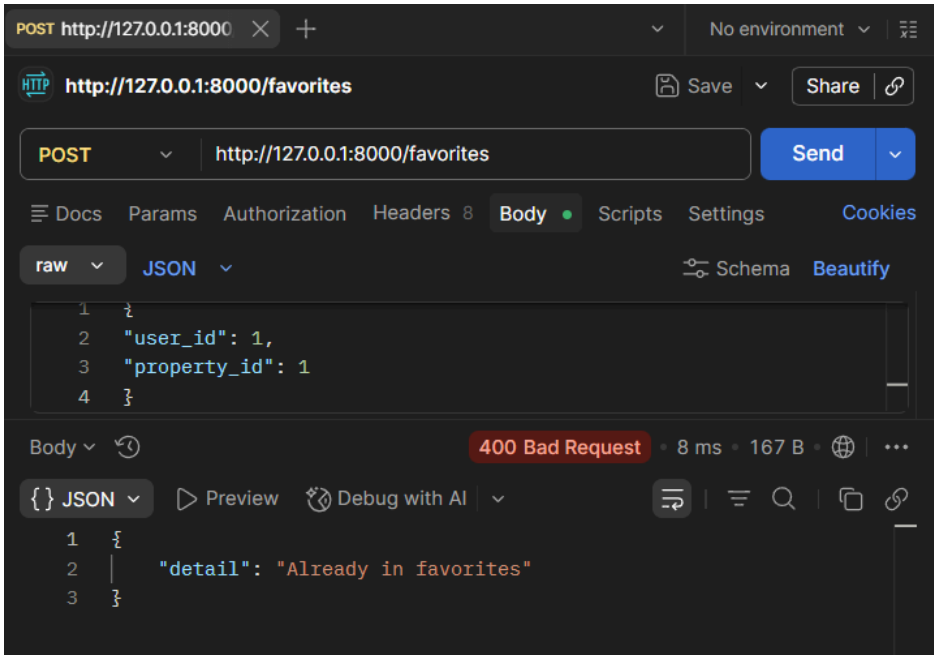
{ } JSON Preview Visualize


```
[
    {
        "title": "Luxury Villa",
        "city": "Delhi",
        "price": 750000.0,
        "agent_id": 1,
        "id": 1
    },
    {
        "title": "Luxury Villa",
        "city": "Delhi",
        "price": 750000.0,
        "agent_id": 1,
        "id": 2
    }
]
```

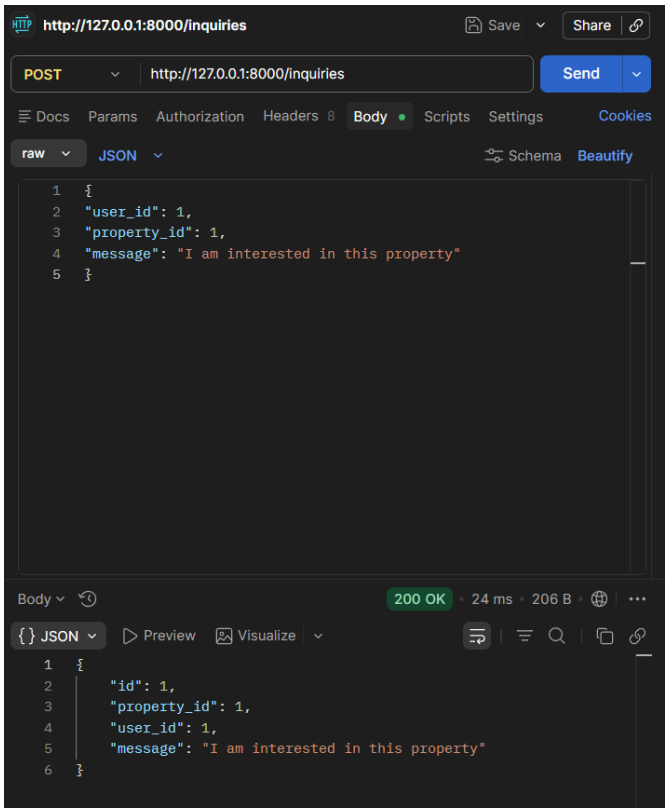
Add favorites:



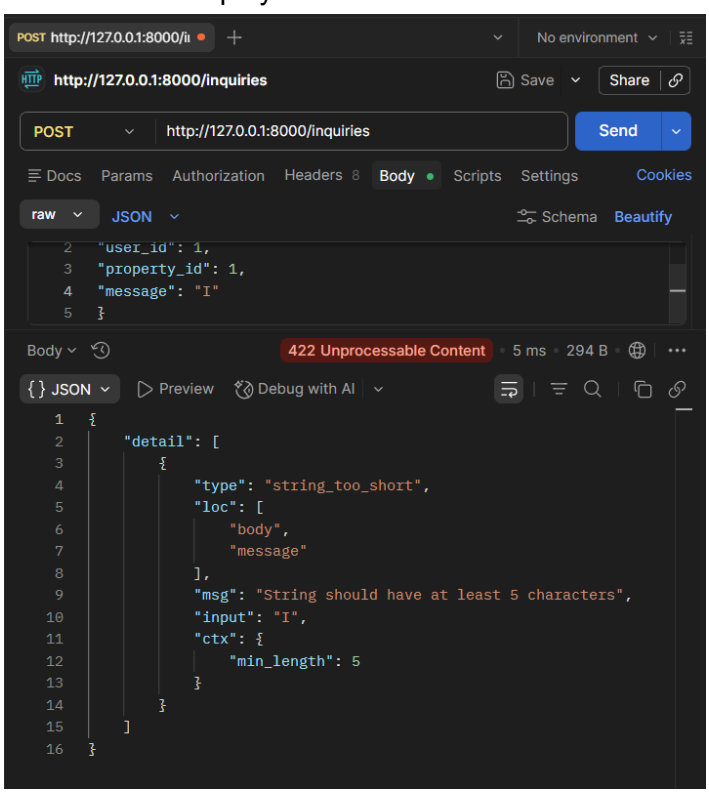

Add favorites that already exists:



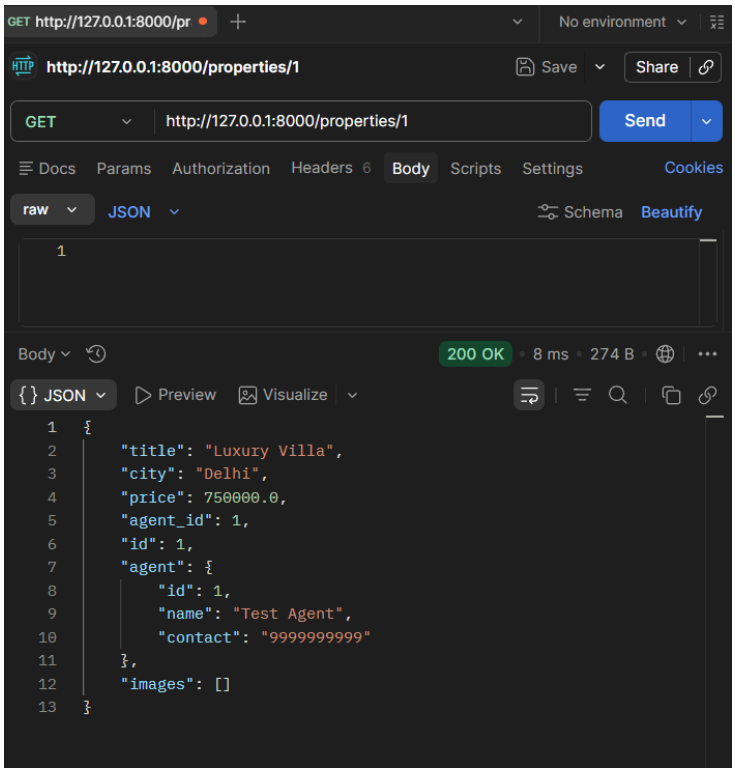
Create inquiry:



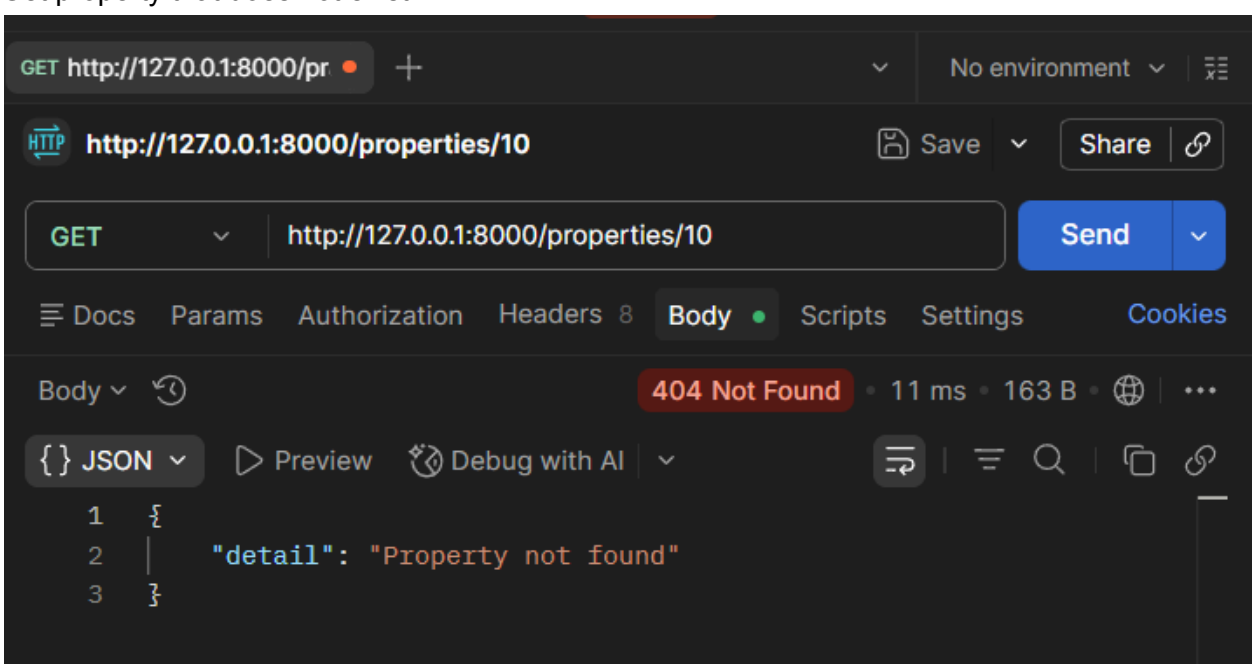

Create invalid inquiry:



Get property details:




Get property that does not exist:



# Real Estate Listing API – Project Summary

1. Developed a **RESTful API using FastAPI** for managing real estate listings, users, and interactions.
2. Designed a **modular architecture** with separate layers for models, schemas, CRUD logic, and routes.

## Core Features

- **Authentication**
  - User registration and login APIs
- **Property Management**
  - Create, update, delete properties
  - List all properties with pagination
  - View detailed property info (agent + images)
- **Search & Filter**
  - Filter properties by city and price range
- **User Interactions**
  - Add favorites
  - Create inquiries for properties

## Database Design

- Used **SQLite with SQLAlchemy ORM**
- Tables:
  - users, agents, properties, property_images, favorites, inquiries
- Implemented **relationships and foreign keys** for data integrity

## Testing

- Created **pytest test cases** for:
  - Authentication (register, login)
  - Property APIs (CRUD, search, details)
  - Favorites & inquiries
- Ensured **all tests pass successfully**
- Handled **edge cases and duplicate data** for repeatable test runs
- Performed manual testing using Postman

## Additional Implementations

- Added **seed data** for immediate testing
- Included **basic validations** (e.g., price, agent existence)
- Ensured **clean API responses and error handling**

## Outcome

- **Fully functional backend system with**:
  - Clean structure
  - Reliable API behavior
  - Complete test coverage
  - Ready for integration or deployment

**END**